EE5203 L09 Study Sheet

ADC and sensor acquisition - Basri Erdogan

Essential question: Every signal so far has been high or low. A knob, a light sensor, a temperature — these are neither. How does a voltage become a number your program can reason about, and how much should you trust it?

What you should be able to do

- Explain resolution, LSB size, and the counts millivolts conversion.
- Configure ADC1 and justify the resolution, sampling time, and mode.
- Read a channel by polling, paced by the L07 tick, and switch channels safely.
- Map counts to a control value in integer arithmetic.
- Verify `DR`, `EOC`, and a live continuous-mode experiment in the SFR view.

Counts and millivolts

$$\text{mV} = \frac{\text{raw} \times 3300}{4095} \qquad \text{LSB} = \frac{3300\ \text{mV}}{4095} \approx 0.806\ \text{mV}$$

`raw`	mV	LED duty (`×999/4095`)	Servo (`1000 + ×1000/4095`)
0	0	0	1000 µs — 0°
1024	825	249	1250 µs — 45°
2048	1650	499	1500 µs — 90°
3072	2475	749	1750 µs — 135°
4095	3300	999	2000 µs — 180°

- **12 bits** means 4096 levels, `0` … `4095`. Full scale (4095) maps to full reference (3300 mV) — the endpoint convention.
- **Integers throughout.** `(raw * 3300u) / 4095u` fits comfortably in `uint32_t`. Multiply *before* dividing, or you lose everything to truncation. Convert to float only at a display edge, if at all.
- **The reference is the ruler.** Every count is relative to VREF. If the 3.3 V rail sags, every reading is wrong in the same direction, and nothing in the data shows it.

Vocabulary table

Name	Job
Resolution	how many levels — 12 bits → 4096
LSB	the voltage one count represents 0.806 mV
Sampling time	how long the input charges the internal hold capacitor
Conversion time	sampling + 12 SAR cycles
`EOC` (`SR`)	end of conversion — a result is waiting
`DR`	data register — reading it clears `EOC`
`CONT`	continuous mode: convert again immediately, forever
SAR	successive approximation — the binary-search guessing game

Configuration and reading

```
/* CubeMX: PA0 -> ADC1_IN0, PA1 -> ADC1_IN1, 12-bit,
   continuous DISABLED, EOC single conversion, 84-cycle sampling. */

uint16_t adc_read(uint32_t ch)
{
    ADC_ChannelConfTypeDef s = {0};
    s.Channel      = ch;
    s.Rank         = 1;
    s.SamplingTime = ADC_SAMPLETIME_84CYCLES;
    HAL_ADC_ConfigChannel(&hadc1, &s);

    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);   /* waits for EOC */
```

```
    uint16_t raw = HAL_ADC_GetValue(&hadc1); /* reads DR, clears EOC */
    HAL_ADC_Stop(&hadc1);
    return raw;
}
```

Pace the reads from the tick, never from `HAL_Delay`:

```
if ((g_ms - t_adc) >= 10U) {            /* L07 tick schedules the read */
    t_adc = g_ms;
    uint16_t raw  = adc_read(ADC_CHANNEL_0);
    uint32_t duty = (raw * 999u) / 4095u;             /* L08's ARR */
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, duty);
}
```

Sampling time. The input must charge the sample-and-hold capacitor through the source's own resistance. A low-impedance source (a pot wiper) charges fast; a high-impedance one (an LDR divider, a thermistor) needs longer. Too short and the reading is dragged toward the *previous* channel's value — the classic symptom of cross-talk when switching channels.

One ADC, many channels. There is one converter behind a multiplexer. To read two channels you reconfigure rank 1 and convert again — which is what `adc_read(ch)` does. Both readings are therefore taken at slightly different moments, not simultaneously.

The pin goes analog

`MODER` for PA0 reads `11` — analog. That is the fourth row of the table L04 introduced, and this is the lecture where it earns its place. In analog mode the digital input buffer is disconnected, which is both why the ADC can see the true voltage and why `HAL_GPIO_ReadPin` on that pin is meaningless.

The three flag families, so far

Flag	Peripheral	Set when
`UIF`	timer (L07)	the counter wrapped at `ARR`
`CC1IF`	timer channel (L08)	an edge was captured
`EOC`	ADC (L09)	a conversion finished; cleared by reading `DR`

Same pattern every time: hardware raises a flag, software tests it or takes an interrupt, and something clears it.

SFR evidence

Claim	Evidence
the pin is analog	`GPIOA->MODER` bits 1:0 = `11`
a conversion completed	`ADC1->SR` `EOC` = 1 before you read `DR`
the result is real	`ADC1->DR` tracks the knob as you turn it
reading clears the flag	`EOC` returns to 0 immediately after `GetValue`
continuous mode really is continuous	set `CONT` in `ADC1->CR2` live and watch `DR` update with no `Start` call

Common mistakes

- Dividing before multiplying: `(raw / 4095) * 3300` is always 0.
- Assuming 5 V reference — the board's analog reference is 3.3 V.
- Sampling time too short for a high-impedance sensor; readings drift toward the previously converted channel.
- Reading `DR` without checking `EOC`, and getting the previous conversion.
- Calling `HAL_ADC_Start` once and expecting fresh values forever in single mode.
- Blocking on `PollForConversion` inside an interrupt callback.
- Wiring the pot's wiper to a rail instead of the middle terminal.
- Expecting an LDR divider to be linear in light — it is not, and it does not need to be for a threshold.

Mastery self-check

- ☐ I can convert counts to millivolts and back, in integers.
- ☐ I can state the LSB size and what resolution buys me.
- ☐ I can justify a sampling time from the source impedance.
- ☐ I can explain why two channels are not sampled simultaneously.
- ☐ I can name what clears `EOC`.
- ☐ I can predict `MODER` for an analog pin.
- ☐ I can map a reading onto a duty and a servo pulse without floats.
- ☐ I can say what happens to every reading if VREF sags.

Five review questions

1. A reading of `2867` arrives on the 12-bit scale. Give the voltage in millivolts, the LED duty it commands via `×999/4095`, and the servo pulse via `1000 + ×1000/4095`.
2. A student writes `mV = (raw / 4095) * 3300` and always gets 0 (or 3300). Explain precisely what integer division did, and give the correct expression.
3. The pot reads correctly, but the LDR channel returns values that look like a faded copy of the pot's. Name the most likely cause and the one setting you would change.
4. Why does reading `ADC1->DR` clear `EOC`, and what goes wrong in a polling loop that reads `DR` twice per conversion?
5. Your night-light switches LD2 on and off rapidly whenever the room light sits near the threshold. Name the phenomenon, and give a fix that does not involve changing the sensor.

See the L09 worksheet for extended practice. Worked solutions are released after the practice window.